

External Shopping Cart

Compatible With Taopix
Control Centre v2021r1+

Author: Taopix Support

Date: 01/04/2021

Document Name: Taopix Web
Shopping Cart Breakout v9.2.1

PRIVATE & CONFIDENTIAL

Revision History

5 th April 2012	v2.0	Updated the client side library functions <code>insertOrderHeader</code> , <code>insertOrderItem</code> , <code>insertOrderItemComponent</code> and <code>insertOrderShipping</code> so that they pass the empty 'extra' parameter.
5 th April 2012	v2.0	Added the parameter 'extra' to the <code>INSERT DataAPI</code> call
26 th March 2012	v2.0	Added information on the new <code>cancelSession</code> API call
26 th March 2012	v2.0	Updated the example code to better demonstrate data being transmitted between Taopix and the external shopping cart.
26 th March 2012	v2.0	Updated the example code to demonstrate inserting orders with multiple line items and external assets.
26 th March 2012	v2.0	Removed the column 'orderid' from <code>insertOrderItem</code> as it is no longer required. Any custom code referencing this column should also be updated.
26 th March 2012	v2.0	Added the columns 'uploadappversion', 'uploadappplatform', 'uploadappcpu' and 'uploadappversion' to <code>insertOrderItem</code> . Integrations must be updated to process these new columns.
26 th March 2012	v2.0	Added the columns 'voucherapplied', 'itemnumber', 'productinfo' and 'assetid' to <code>insertOrderItem</code> . Integrations must be updated to process these new columns.
26 th March 2012	v2.0	Added the columns 'itemcount', 'shippingtotalsellbeforediscount', 'showzerotax', 'showtaxbreakdown' and 'showalwaysstaxtotal' to <code>insertOrderHeader</code> . Integrations must be updated to process these new columns.
30 th April 2012	v2.0	Added the column 'pricingmodel' to <code>insertOrderItemComponent</code> . Integrations must be updated to process this new column.
30 th April 2012	v2.0	Documented how to add Taopix Library Link external assets into the database.
30 th April 2012	v2.0	Updated the information on limitations of the external shopping cart integration.
30 th April 2012	v2.0	Added a section on upgrading from previous versions of the API.
30 th April 2012	v2.0	Added a section covering the sample code provided with the API.
21 st October 2013	v3.0	Updated the documentation to cover both the Desktop Designer and Online Designer.
21 st October 2013	v3.0	Added the column 'source' to <code>insertOrderItem</code> . Integrations must be updated to process these new columns.
21 st October 2013	v3.0	Added information relating to the user account information that could be passed through from Taopix.
21 st October 2013	v3.0	Added a section relating to user authentication.
21 st October 2013	v3.0	Updated the information relating to inserting and updating customer accounts.
21 st October 2013	v3.0	Updated the information relating to address formats.
17 th January 2014	v4.0	Updated the section relating to checking the shopping cart status to mention the new <code>CancelOrder</code> API.
17 th January 2014	v4.0	Updated the sections relating to the <code>DeleteSession</code> API to mention the new <code>CancelOrder</code> API.
17 th January 2014	v4.0	Added information on the new <code>CancelOrder</code> API call.
17 th January 2014	v4.0	Updated the example code to use the new <code>CancelOrder</code> API call.
4 th July 2014	v5.0	Updated the example to use userid instead of uuid when inserting an order.
4 th July 2014	v5.0	Updated the example to state that the userid should be assigned.
4 th July 2014	v5.0	Updated the example to assign the userid to all order database records.
16 th December 2014	v6.0	Updated the terminology used within the documentation.
16 th December 2014	v6.0	Updated the URLs within the sample code.
16 th December 2014	v6.0	Updated the sample code installation steps.
2 nd July 2015	v7.0	Updated the section related to the initialisation process to include single prints data.
2 nd July 2015	v7.0	Updated the section related to the <code>checkPageCount</code> API to include single prints data.
2 nd July 2015	v7.0	Updated the section related to inserting components.
2 nd July 2015	v7.0	Added a section related to inserting sub-components.
2 nd July 2015	v7.0	Added a section related to inserting single prints.
2 nd July 2015	v7.0	Updated the section relating to upgrading from previous versions.
2 nd July 2015	v7.0	Added the column <code>parentcomponentid</code> to <code>insertOrderItem</code> . Integrations must be updated to send a zero value if sub-components are not required.
2 nd July 2015	v7.0	Updated the example to demonstrate inserting sub-components into the database.
2 nd July 2015	v7.0	Updated the example to demonstrate inserting pictures (single prints) into the database.
2 nd July 2015	v7.0	Updated the section related to the <code>checkPageCount</code> API to include external asset and picture data.
2 nd July 2015	v7.0	Updated the initialise API to document the structure of the picture (single prints) data.
2 nd July 2015	v7.0	Updated the <code>checkPageCount</code> API to include external picture (single prints) data.
09 th Feb 2016	v8.0	Added a section related to inserting calendar customisation.
2 nd September 2016	v8.0.1	Updated example <code>clientSideLibrary::apiCall</code> method names.
1 st August 2019	v9.0.0	Taopix Data API separated out into its own Taopix Data API v11.0.0 document. Added the example <code>Taopix_External_Shopping_Cart_SampleV2.php</code> which demonstrates a simplified method of inserting an order into Taopix using the Data API.
3 rd December 2019	V9.1.0	Added support for AI Pricing
4 th February 2020	V9.1.0	Added optional 'sitecode' column on ORDERITEMS.

27 th May 2020	V9.2.0	Added 'projectaimode' column on ORDERITEMS.
1 st April 2021	V9.2.1	Updated initialise API input parameters to include 'projectpreviewthumbnail' in 'items[]' for displaying the project preview thumbnail or to be downloaded and stored.

INTRODUCTION

The external shopping cart API is a licensable feature within Taopix Control Centre that allows the built-in shopping cart to be bypassed and replaced by an external shopping cart. As each external shopping cart will perform differently it is not possible to build a direct link within Taopix Control Centre. Instead, an API is available within Taopix Control Centre that will allow 3rd party integrators to link the two systems together.

The API consists of the following:

1. An external PHP script running on the Taopix Control Centre server that will allow the interaction between Taopix Designer, Taopix Control Centre and the external shopping cart to take place.
2. A HTTP POST / JSON based API that will allow the external shopping cart to populate the Taopix database with the part of the order that is relevant to Taopix.
3. A HTTP POST / JSON based API that will allow the external shopping cart to inform Taopix that the order process has been abandoned.
4. A HTTP POST / JSON based API that will allow the external shopping cart (or other system) to provide status / workflow updates to the Taopix system.

NOTE

Taopix Control Centre will still be used to handle the auto-update process within the Taopix Desktop Designer and to specify which products are available to both the Desktop and Online designers. This means that there is still a requirement to configure basic product pricing to link products to license keys. This does, however, have the advantage of allowing different brands to use different shopping carts for the same product ranges.

TAOPIX CONTROL CENTRE CONFIGURATION REQUIREMENTS

Although the external shopping cart handles all aspects of the shopping cart experience there is still a requirement to configure certain parts of the Taopix system.

1. Taopix Creator System Upload

The system upload procedure from Taopix Creator will still be performed in the exact same way.

2. Taopix Control Centre Application Asset Administration

The administration of application assets (masks, backgrounds etc...) will still be performed in the exact same way.

3. Taopix Control Centre License Key Administration

The administration of license keys will still be required although some settings will not be relevant when the Taopix shopping cart is bypassed.

4. Taopix Control Centre Product Collection / Product Administration

The administration of product collections and products within Taopix Control Centre (making them active etc...) will still be performed in the exact same way.

5. Taopix Control Centre Pricing

Although the external shopping cart will handle all pricing there will still be a requirement to create active pricing to link products to license keys / brands. If pricing is to be displayed in the Taopix Designer then the products and components must contain correct pricing. If pricing is not displayed in the Taopix Designer then the content pricing may not be required.

6. Taopix Control Centre Brand Administration

Brand administration can still be performed if you require license keys to be grouped into separate brands for auto-update purposes or for use with the external script. Any branding changes (such as logo's etc...) made within the Taopix Control Centre will not have any impact on the external shopping cart. This means that it is the responsibility of the external shopping cart to implement any appearance changes based on the brand.

TAOPIX LICENSEE SOFTWARE DEVELOPMENT / SKILLS REQUIREMENTS

1. PHP

There will be a requirement for the creation of a custom PHP script that will handle the interaction between Taopix Designer, Taopix Control Centre and the external shopping cart. The contents of this script and skills required will depend on what is required to communicate with the external shopping cart. More information on the script is provided later in this document.

2. External Shopping Cart Development

As the Taopix shopping cart will be bypassed there will be a requirement for the external shopping cart to support the interaction between the systems and all aspects of the shopping cart experience. It is expected that this will require some modifications to the external shopping cart by the licensee thus needing development skills in the programming language that was used to create it. Some areas that may need implementing are:

a) Shopping Cart Experience

- Component selection (cover / paper etc...)
- Pricing
- Tax calculations
- Shipping calculations
- Payment capture
- Branded appearance

b) Order Creation

When an order is created within the external shopping cart part of this order will be relevant to Taopix. The external shopping cart must be modified to perform the process of inserting this part of the order into the Taopix database. The interaction between the external shopping cart and Taopix Control Centre is performed via a HTTP based JSON API. This API uses Blowfish encryption to help prevent hacking and man-in-the-middle attacks.

c) Job Ticket Cancellation

If the customer ends the shopping process without completing an order it is possible for the external shopping cart to notify Taopix Control Centre via the JSON API.

d) Customer Notification

The external shopping cart will be responsible for handling all customer notifications when an order is created.

d) Back Office

Depending on the requirements of the system it may be necessary for the back office to update Taopix Control Centre with status updates such as the order being shipping / completed / cancelled etc...

Additional Information

1. Taopix uses UTF-8 as it's native text encoding. If the external shopping cart does not use UTF-8 it is the responsibility of the integrator to perform any relevant re-encoding when communicating between Taopix and the external shopping cart.
2. Taopix stores various names and descriptions for multiple languages in a single string known as the Taopix multi-lingual string format. The format of a multi-lingual string is the language code followed by a space and then the text for the language. Strings that contain multiple languages are separated by the paragraph <p> tag.

Examples:

en this is the English description

en this is the English description<p>de this is the German description

Skills / Technology Summary

1. PHP (v5)
2. Programming language used to create external shopping cart
3. JSON
4. HTTP communication
5. Blowfish encryption that supports CBC mode and an initialisation vector
6. Knowledge of Unicode / UTF-8
7. Basic operational knowledge of Taopix Designer and Taopix Control Centre

NOTES

1. It is the responsibility of Taopix Limited to provide and support a working script interface and API. It will not be the responsibility of Taopix Limited to perform or support any of the integration work required by the licensee.
2. Although there is a requirement for PHP development skills to create the script that interacts between the systems there is no requirement for the external shopping cart to use PHP.
3. The Taopix system is constantly evolving and it will be necessary for the licensee to update the integration as new features and enhancements are implemented.

Sample Code

The API comes complete with an example that demonstrates the key concepts of external shopping cart integration and the Data API. The sample code consists of the following files:

1. EDL_ExternalShoppingCart.php

This contains example glue code between Taopix Control Centre and the external shopping cart. The file provided currently contains code to interact with the API test code.

2. Taopix_External_Shopping_Cart_SampleV1.php

This file contains the API test code to simulate an external system inserting an order into the Taopix database. This example demonstrates how an order can be inserted into the Taopix database utilizing all of the Data API endpoints.

3. Taopix_External_Shopping_Cart_SampleV2.php

This file contains the API test code to simulate an external system inserting an order into the Taopix database. This is a simplified example and demonstrates how an order can be inserted using new Data API endpoints `getProjectOrderData` and `insertOrder`. **Note: This is the recommended method for new customers.**

4. clientSideLibrary.php

This file contains various functions that are used by the script that simulates an external system inserting an order into the database using the Taopix Data API.

5. dataapi.conf

This file contains the config settings used by the API sample.

DATAAPISECRETKEY

This is the data API secret which must match the one within the *mediaalbumweb.conf* file.

SERVERURL

This is the url to the Taopix Control Centre server

USERNAME

This is the API user login configured within Taopix Control Centre

PASSWORD

This is the API user password configured within Taopix Control Centre

6. mediaalbumweb.conf additional

This file contains additional config settings for *mediaalbumweb.conf*.

DATAAPISECRETKEY

This is the data API secret that must match the one within the *dataapi.conf* file.

DATAAPIENCRYPTRESULT

This instructs the data API to decrypt the result from the server (value should match the `kOutputIsEncrypted` constant within *clientSideLibrary.php*).

Sample Code Installation / External Shopping Cart Configuration

1. Create a new API type user login within Taopix Control Centre.
2. Update *mediaalbumweb.conf* with the Data API settings.
3. Copy *EDL_ExternalShoppingCart.php* into *customiselscripts*
4. Update *EDL_ExternalShoppingCart.php* with the URL to the sample cart url (eg: the URL to *Taopix_External_Shopping_Cart_SampleV1.php* or *Taopix_External_Shopping_Cart_SampleV2.php*).
5. Update *dataapi.conf* with the relevant Data API settings and URL to Taopix Control Centre.
6. Copy the sample cart files onto a web server.
7. Edit a Product price within Taopix Control Centre and tick the *Use External Shopping Cart* checkbox.

When the *External Shopping Cart* option is ticked, Taopix Control Centre will divert to the external shopping cart when the relevant product / license key combination is found. If a multi-line order is being initiated, Taopix Control Centre will use the product on the first line to determine if the external shopping cart should be used.

NOTE

The example code uses PHP session IDs within the URL to transfer the data between Taopix Control Centre and the sample cart. For the example to work correctly it may be necessary to update your *php.ini* file with the following:

```
session.use_cookies = 0  
session.use_only_cookies = 0
```

Supplying the session ID on the URL is only to simplify the example code. It is neither a requirement or recommendation on how this process should be performed in real-life live integration.

UPGRADING FROM PREVIOUS VERSIONS

Upgrading to a newer version of the External Shopping Cart / Data API will usually involve making changes to your integration code. Depending on the enhancements made to the system this could involve changes to one of the following areas:

1. Your implementation of `EDL_ExternalShoppingCart.php`.
2. Changes to the external shopping cart either to store additional data provided by the API or to implement new functionality.
3. Changes to your code that integrates with the Taopix Data API to insert an order into the Taopix database.

The first steps in this process should be to always thoroughly read this document and the sample code that is provided as this should provide an understanding of the API changes and if they effect your integration.

If the documentation states that new columns have been added or that columns have been removed then there is a high probability that your integration will need to be updated to work with the version of Taopix. If the changes are related to specific Taopix features (such as Library Link) then it may not be necessary to update your integration if you are not taking advantage of those features.

Finally, always test any changes to your integration in a staging environment using different versions of Taopix Designer to make sure that your integration continues to operate correctly.

INITIAL COMMUNICATION

The initial communication occurs when Taopix Designer communicates with Taopix Control Centre at the start of the shopping cart workflow. When this occurs, Taopix Control Centre initiates a web session that is communicated back to Taopix Designer in the form of a HTTP URL. This URL is then opened within the customer's web browser.

When a product pricelist / license key combination states that the product should be handled by the external shopping cart, Taopix Control Centre will first create a web session as normal but instead of returning with the URL of the Taopix shopping cart it will, instead, call and pass the initialisation parameters to the `initialise` function within the external script `EDL_ExternalShoppingCart.php`.

The input to the function is a single PHP associative array that receives all of the parameters to be transferred to the external shopping cart (see below for a list of parameters). The output of this function is also a PHP associative array that communicates the result of the initialisation back to Taopix Control Centre.

```
function initialise($pParamArray)
{
    $result = Array();

    // perform initialisation

    // return the response of the initialisation
    $result['result'] = '';
    $result['usecustomshoppingcart'] = true;

    $result['shoppingcarturl'] = '';

    return $result;
}
```

The functions that the integration should perform at this stage are:

- a) The script must transmit all of the parameters to the external shopping cart in a format that can be processed by the external shopping cart.
- b) The external shopping cart must receive and store all of the parameters. As of 2019r2 all of the parameters will be stored in the control centre database. These parameters can be retrieved by making use of the new `getProjectOrderData` Data API endpoint.
- c) The external shopping cart must use any relevant parameters to initialise it's own shopping cart session (ie: product code, group code, brand code).
- d) The external shopping cart should automatically authenticate the customer if they have already authenticated within Taopix and the account login information has been passed through (see later in this document for more information).
- e) The external shopping cart must return a response that can be decoded by the script and returned back to Taopix Control Centre.

The response from the external shopping cart can be in any format as long as the script can decode it and pass the relevant information back to Taopix Control Centre as a PHP associative array with the following values:

result	string	An error message to be returned back to Taopix Designer or an empty string if no error occurred.
usecustomshoppingcart	boolean	A Boolean that indicates whether the external shopping cart should be used (true) or if the system should revert back to the Taopix shopping cart (false)
shoppingcarturl	string	A string containing a web page URL for the external shopping cart session

The sample code provided demonstrates one way to transmit the initialisation data by serialising the parameter structure, compressing it, base 64 encoding it and then passing it to the sample cart script.

```
define('kExternalShoppingCartURL', 'http://external.cart.url/Taopix_External_Shopping_Cart_SampleV1.php');

function initialise($pParamArray)
{
    $result = Array();

    /*
     * here is where the external shopping cart would be initialised
     * this would be performed by an api call to the external shopping cart that initialises a session
     * using the supplied parameters
     * and then returns a url to the external cart session
     */

    /*
     * in this example we serialize the array into a string, compress it and then base64encode it so we
     * can post it as a parameter
     */
    $paramData = serialize($pParamArray);
    $paramDataLength = strlen($paramData);
    $paramData = base64_encode(gzcompress($paramData, 9));

    $postParamArray = Array('cmd' => 'init', 'paramdata' => $paramData, 'paramdatalen' =>
    $paramDataLength);
    $postResult = self::cURLPost(kExternalShoppingCartURL, $postParamArray);

    // return the response of the initialization
    $result['result'] = '';
    $result['usecustomshoppingcart'] = true;

    $result['shoppingcarturl'] = $postResult;

    return $result;
}

function cURLPost($pURL, $pParamArray)
{
    /*
     * convert the parameters for the POST
     */
    $paramString = http_build_query($pParamArray);

    /*
     * configure the connection
     */
    curl_setopt($ch, CURLOPT_URL, $pURL);
    curl_setopt($ch, CURLOPT_SSL_VERIFYPEER, FALSE);
    curl_setopt($ch, CURLOPT_HEADER, 0);
    curl_setopt($ch, CURLOPT_TIMEOUT, 30);
    curl_setopt($ch, CURLOPT_POST, 1);
    curl_setopt($ch, CURLOPT_POSTFIELDS, $paramString);
    curl_setopt($ch, CURLOPT_RETURNTRANSFER, 1);

    /*
     * execute post
     */
    $result = curl_exec($ch);

    if ($result)
    {
        /*
         * we have a result so trim the data before the result= tag
         */
        $result = substr($result, strpos($result, '=') + 1);
    }
    else
    {
        error_log(curl_error($ch));
    }

    /*
     * close connection
     */
    curl_close($ch);

    return $result;
}
```

Input Parameters

The input parameters provided to the external script are as follows:

apiversion	integer	Taopix Designer API version
languagecode	varchar(50)	Taopix Designer language code
ownercode	varchar(50)	Taopix System unique identifier
groupcode	varchar(50)	Taopix license key code
groupdata	varchar(50)	Taopix license key
brandcode	varchar(50)	Taopix brand code
userid	integer	Taopix user record id if user has previously authenticated within Taopix or 0 if the user has not authenticated within Taopix.
userlogin	varchar(50)	Taopix user record login if user has previously authenticated within Taopix or an empty string if the user has not authenticated within Taopix.
useraccountcode	varchar(50)	Taopix user record account code if user has previously authenticated within Taopix or an empty string if the user has not authenticated within Taopix.
userstatus	integer	Taopix user record status if the user has authenticated within Taopix: 0=Taopix would like the external cart to prompt the customer with their current account details asking them to check and update all relevant information. Afterwards, the external cart should then update the user record <i>userid</i> within the Taopix database. 1=Customer account information requires no update. 2=Only the minimum account information was recorded within Taopix when the user registered. Taopix would like the external cart to prompt the customer with their current account details asking them to check and update all relevant information. Afterwards, the external cart should then update the user record <i>userid</i> within the Taopix database.
uuid	varchar(200)	Taopix Designer unique identifier
ref	integer	Taopix Control Centre session reference
origorderid	integer	The original order header id if this is a re-order
origordernumber	varchar(50)	The original order number if this is a re-order
shoppingcarturl	varchar(1024)	The URL that would be used to show the Taopix shopping cart
batchref	varchar(200)	The batch reference that will group the order items together within the database
reorder	integer	Status indicating if this is a re-order 0=no 1=yes
items[]	array	A PHP associate array that will contain one or more projects to be ordered

Each item will contain the following values:

projectref	varchar(300)	The unique identifier for the project
projectreforig	varchar(300)	The unique identifier for the project that this project was originally based on (duplicated from)
projectname	varchar(200)	The project name
shareid	integer	The record id from the Taopix shared items table if the order was initialised via clicking a send-to-a-friend link
source	integer	The source of the order item (desktop designer or online designer)
origorderitemid	integer	The original order item record id if this is a re-order
uploadgroupcode	varchar(50)	The Taopix license key code the project was originally uploaded against
uploadorderid	integer	The Taopix order header record id the project was originally uploaded against
uploadordernumber	varchar(50)	The Taopix order number the project was originally uploaded against
uploadorderitemid	integer	The Taopix order item record id the project was originally uploaded against
uploadref	varchar(200)	The unique upload reference for the project
collectioncode	varchar(50)	The product collection code
collectionname	varchar(1024)	The product collection name in Taopix multi-lingual string format
productcode	varchar(50)	The product code
productname	varchar(1024)	The product name in Taopix multi-lingual string format
producttype	integer	The project's product type 0=photobook 1=proofbook 3=calendar
productpageformat	integer	The project's product page format 0=single pages 1=spreads
productsreadformat	integer	The project's product spread format
productcover1format	integer	The project's product cover 1 format 0=no cover 1=front cover 2=wrap-around cover
productcover2format	integer	The project's product cover 2 format 0=no cover 1=cover (either back or wrap-around depending on <i>productcover1format</i>)
productoutputformat	integer	The project's product output format 0=output as single pages 1=output as spreads
productheight	decimal(10, 6)	The project's product page height in inches
productwidth	decimal(10, 6)	The project's product page width in inches
productskucode	varchar(50)	The product SKU code as defined within Taopix Control Centre
productunitcost	decimal(10, 2)	The product unit cost as defined within Taopix Control Centre
productunitweight	decimal(10, 4)	The product unit weight as defined within Taopix Control Centre
productdefaultpagecount	integer	The product's default page count as

		defined within Taopix Control Centre
pagecount	integer	The number of pages within the project
uploadappversion	varchar(20)	The Taopix version used to upload this project
uploadappplatform	varchar(20)	The operating system platform used to upload this project
uploadappcputype	varchar(20)	The computer processor type used to upload this project
uploadapposversion	varchar(50)	The operating system used to upload this project
externalassets[]	array	A PHP associate array that will contain zero or more external picture assets for the project
pictures[]	array	A PHP associate array that will contain one or more single prints/photo prints picture data for projects using the single prints or photo prints module.
calendarcustomisations[]	array	A PHP associate array that will contain one or more calendar customisation data for a calendar project.
aicomponent[]	Array	A PHP associate array that will contain the data for an AI Component
projectpreviewthumbnail	string	The URL to the project preview thumbnail if the project has one. This URL is signed so will expire so it is recommended to download and stored elsewhere. The URL will expire after the time defined by the URLEXPIRES Taopix Online config setting.

Each asset will contain the following values:

servicecode	varchar(50)	The service code for the asset management system
servicename	varchar(1024)	The service name in Taopix multi-lingual string format
id	varchar(100)	The unique asset ID
name	varchar(1024)	The asset name in Taopix multi-lingual string format
pricetype	integer	0=Free of charge 1=Charged once per project 2=Charged each time used
unitcost	decimal(10, 2)	The unit cost of the asset
unitsell	decimal(10, 2)	The unit sell price of the asset
charge	decimal(10, 2)	The unit charge for the asset taking into account the <i>pricetype</i>
expirationdate	datetime	The asset expiration date
pageref	integer	The project internal page reference number containing the asset
pagenumber	integer	The project page number containing the asset
pagename	varchar(1024)	The project page name containing the asset in Taopix multi-lingual string format
boxref	integer	The project page object reference containing the asset

Each picture will contain the following values:

setid	integer	The ID of the set containing the single
-------	---------	---

		print or photo print or 0 if the item is not in a set
setname	varchar(200)	The name of the set containing the single print or photo print or empty if not in a set
componentqty	integer	The quantity of the single print or photo print
componentcategory	varchar(50)	The single print component category code or photo print
componentcode	varchar(50)	The single print or photo print component code
componentname	varchar(1024)	The single print or photo print component name in Taopix multi-lingual string format
unitcost	decimal(10, 2)	The unit cost of the single print or photo print component from the Taopix pricing table
unitweight	decimal(10, 4)	The unit weight of the single print or photo print component from the Taopix pricing table
unitsell	decimal(10, 2)	The unit sell of the single print or photo print component from the Taopix pricing table
subcomponentcategory	varchar(50)	The single print or photo print sub-component category code
subcomponentcode	varchar(50)	The single print or photo print sub-component code
subcomponentname	varchar(1024)	The single print or photo print sub-component name in Taopix multi-lingual string format
subcomponentunitcost	decimal(10, 2)	The unit cost of the single print sub-component from the Taopix pricing table
subcomponentunitweight	decimal(10, 4)	The unit weight of the single print or photo print sub-component from the Taopix pricing table
subcomponentunitsell	decimal(10, 2)	The unit sell of the single print or photo print sub-component from the Taopix pricing table
assetservicecode	varchar(50)	The service code for the asset management system if this relates to an external asset
assetservicename	varchar(1024)	The service name in Taopix multi-lingual string format if this relates to an external asset
assetpricetype	integer	The price type if this relates to an external asset 0=Free of charge 1=Charged once per project 2=Charged each time used
assetid	varchar(100)	The unique asset ID if this relates to an external asset
assetcost	decimal(10, 2)	The unit cost of the asset if this relates to an external asset
assetsell	decimal(10, 2)	The unit sell price of the asset if this relates to an external asset
pageref	integer	The project internal page reference number containing the picture
pagenumber	integer	The project page number containing the picture

pagename	varchar(1024)	The project page name containing the picture in Taopix multi-lingual string format. This is the single print or photo print picture name.
boxref	integer	The project page object reference containing the picture

Each calendar customisation will contain the following values:

componentqty	integer	The quantity of the calendar customisations
componentcategory	varchar(50)	The calendar customisation component category code
componentcode	varchar(50)	The calendar customisation component code
componentname	varchar(1024)	The calendar customisation component name in Taopix multi-lingual string format
unitcost	decimal(10, 2)	The unit cost of the calendar customisation component from the Taopix pricing table
unitweight	decimal(10, 4)	The unit weight of the calendar customisation component from the Taopix pricing table
unitsell	decimal(10, 2)	The unit sell of the calendar customisation component from the Taopix pricing table

The AI Component will contain the following values

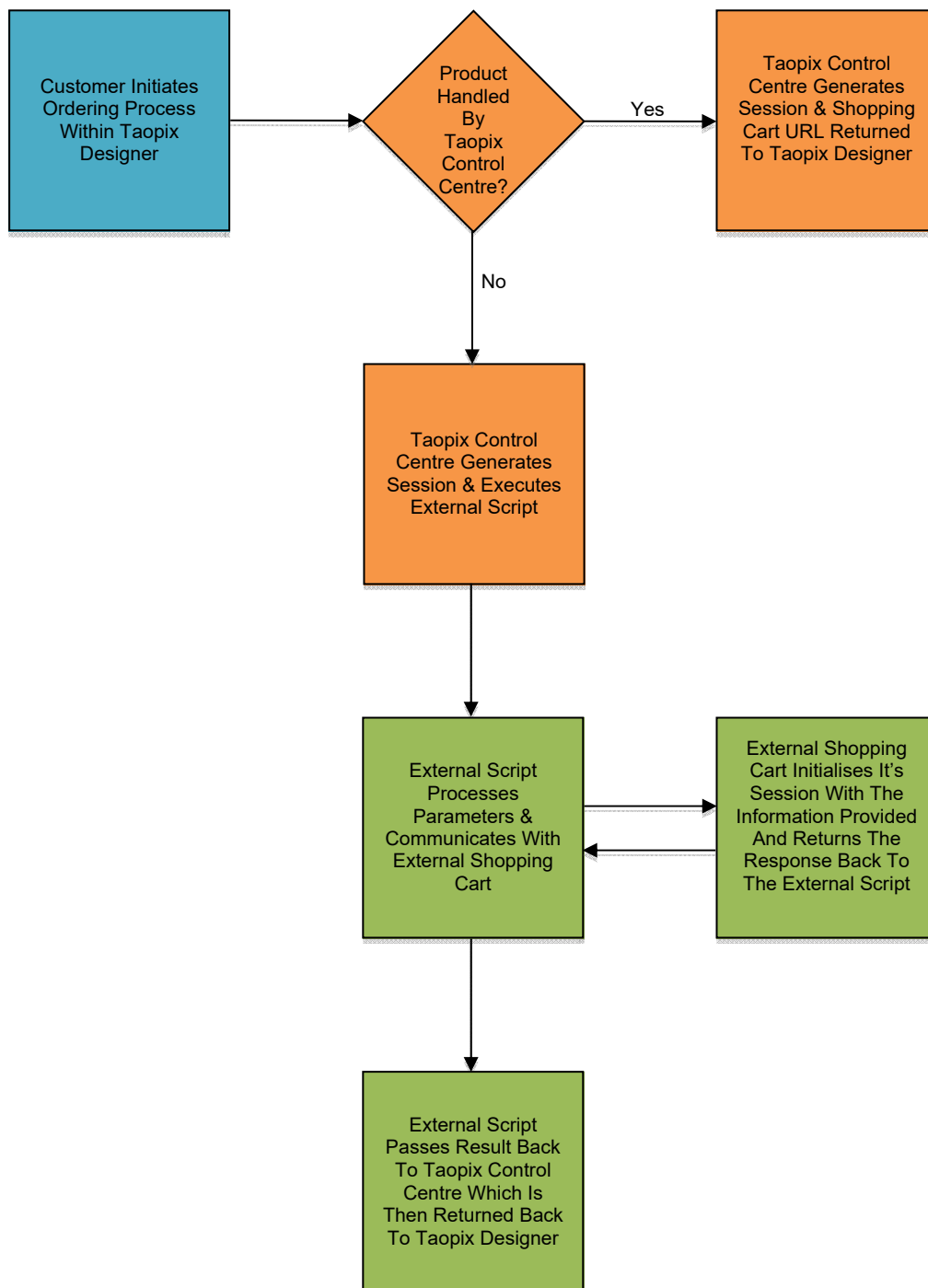
componentqty	integer	The quantity of the AI Component
componentcategory	varchar(50)	The AI Component category code
componentcode	varchar(50)	The AI Component code
componentname	varchar(1024)	The AI Component name in Taopix multi-lingual string format
unitcost	decimal(10, 2)	The unit cost of the AI Component from the Taopix pricing table
unitweight	decimal(10, 4)	The unit weight of the AI Component from the Taopix pricing table
unitsell	decimal(10, 2)	The unit sell of the AI Component from the Taopix pricing table
skucode	Varchar(50)	The SKU code of the AI Component

Customer Authentication

With Taopix version 4.1.0 and later the customer may have already authenticated within Taopix before reaching the shopping cart. Rather than asking the customer to sign-in again the external shopping cart should use the *webbrandcode*, *groupcode*, *userid*, *userlogin*, *useraccountcode* and *userstatus* information to automatically sign the customer into the external shopping cart. As well as providing a much simpler experience for the customer it also prevents problems that could occur if the customer signed into an account within the shopping cart that had no relation to the customer account within Taopix.

If the customer authentication information is being honoured the *userstatus* may also need to be taken into account. This data informs the external shopping cart the state of the customer account information that Taopix holds in it's database and if Taopix thinks that the external cart should prompt the customer to update their details.

Workflow Diagram



-  Process Performed By Taopix Designer
-  Process Performed By Taopix Control Centre
-  Process Performed By External Script / External Shopping Cart (Development Work Required)

SHOPPING CART STATUS (DESKTOP DESIGNER ONLY)

While the customer is interacting with the external shopping cart, Taopix Desktop Designer will communicate with Taopix Control Centre on a regular basis to check the status of the session. Taopix Control Centre will handle the initial processing of this request and if the session is still active it will call the `checkStatus` function within the external script `EDL_ExternalShoppingCart.php`.

The input to the function is a single PHP associative array that contains the following values:

apiversion	integer	Taopix Designer API version
languagecode	varchar(50)	Taopix Designer language code
ownercode	varchar(50)	Taopix System unique identifier
groupcode	varchar(50)	Taopix license key code
brandcode	varchar(50)	Taopix brand code
sessionid	integer	Taopix Control Centre session reference

The output of this function is a PHP string that indicates the next action to be performed by Taopix Control Centre:

ORDER	The shopping cart session is still active
CANCEL	The shopping cart session should be cancelled

```
function checkStatus($pParamArray)
{
    $result = 'ORDER';

    /*
    here is where the external shopping cart would be checked to make sure the external session has not
    been aborted.
    */

    $postParamArray = Array('cmd' => 'status', 'taopixsessionid' => $pParamArray['sessionid']);
    $result = self::cURLPost(kExternalShoppingCartURL, $postParamArray);

    return $result;
}
```

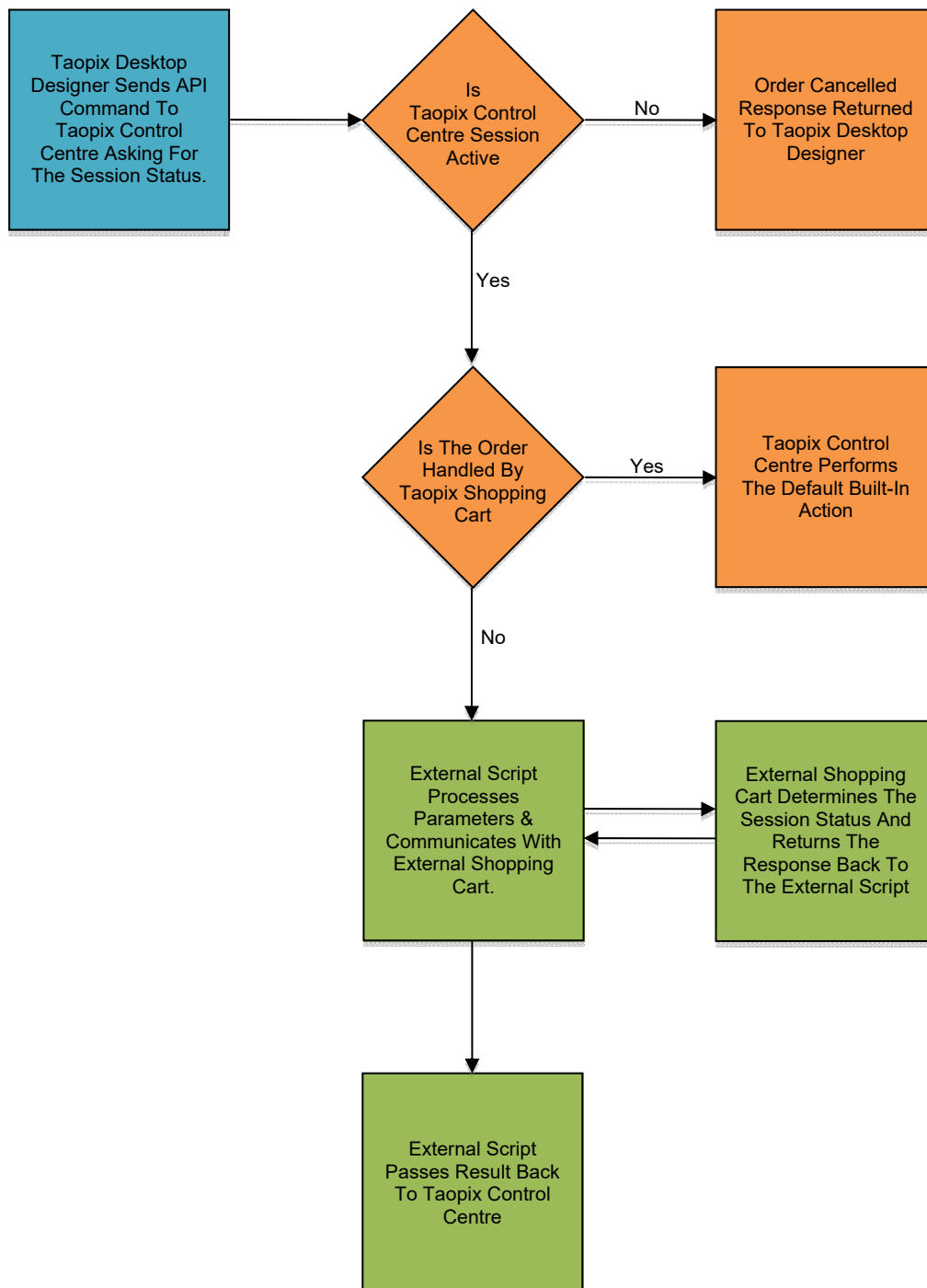
The functions that the integration should perform at this stage are:

- The script should transmit at minimum the `sessionid` value to the external shopping cart.
- The external shopping cart should receive the `sessionid` and use it to identify it's own session.
- The external shopping cart should make a decision whether to let the session continue or to cancel the session and transmit this back to the external script so that it can be returned back to Taopix Control Centre.

NOTE

Desktop Designer version 4.2.0 onwards uses a much more efficient method to check the status. This results in the `checkStatus` function being called less often. One side effect of this change is that if the order process is cancelled within the external shopping cart the Desktop Designer may not detect it for a period of time. To solve this problem a new *CancelOrder* API has been introduced which should be called by the external shopping cart when it's session is no longer valid. Integrations that currently use the *DeleteSession* API in this situation should update their code to use the new *CancelOrder* API. Integrations that currently do not implement this feature should implement it before the v4.2.0 Desktop Designer is deployed to customers.

Workflow Diagram



DESKTOP DESIGNER CANCELLING SESSION

While the customer is interacting with the external shopping cart they have the ability to cancel the process within Taopix Desktop Designer. In Desktop Designer version v3.2.0 and later clicking the Cancel button will now send an API call to Taopix Control Centre requesting the session to be cancelled. If the session is still active Taopix Control Centre will call the `cancelSession` function within the external script `EDL_ExternalShoppingCart.php`.

The input to the function is a single PHP associative array that contains the following values:

<code>apiversion</code>	<code>integer</code>	Taopix Designer API version
<code>languagecode</code>	<code>varchar(50)</code>	Taopix Designer language code
<code>groupcode</code>	<code>varchar(50)</code>	Taopix license key code
<code>brandcode</code>	<code>varchar(50)</code>	Taopix brand code
<code>sessionid</code>	<code>integer</code>	Taopix Control Centre session reference
<code>forcecancel</code>	<code>integer</code>	0=This is a request to cancel the session 1=This session must be cancelled

The output of this function is a PHP boolean that indicates the next action to be performed by Taopix Designer:

<code>false</code>	Prompt the user warning them that they shouldn't cancel at this stage
<code>true</code>	Cancel the session

```
function cancelSession($pParamArray)
{
    $result = true;

    /*
    Here is where the external shopping cart would process the cancellation request.
    If 'forcecancel' is equal to 1 the external shopping cart session should be cancelled immediately
    and the result set to true.

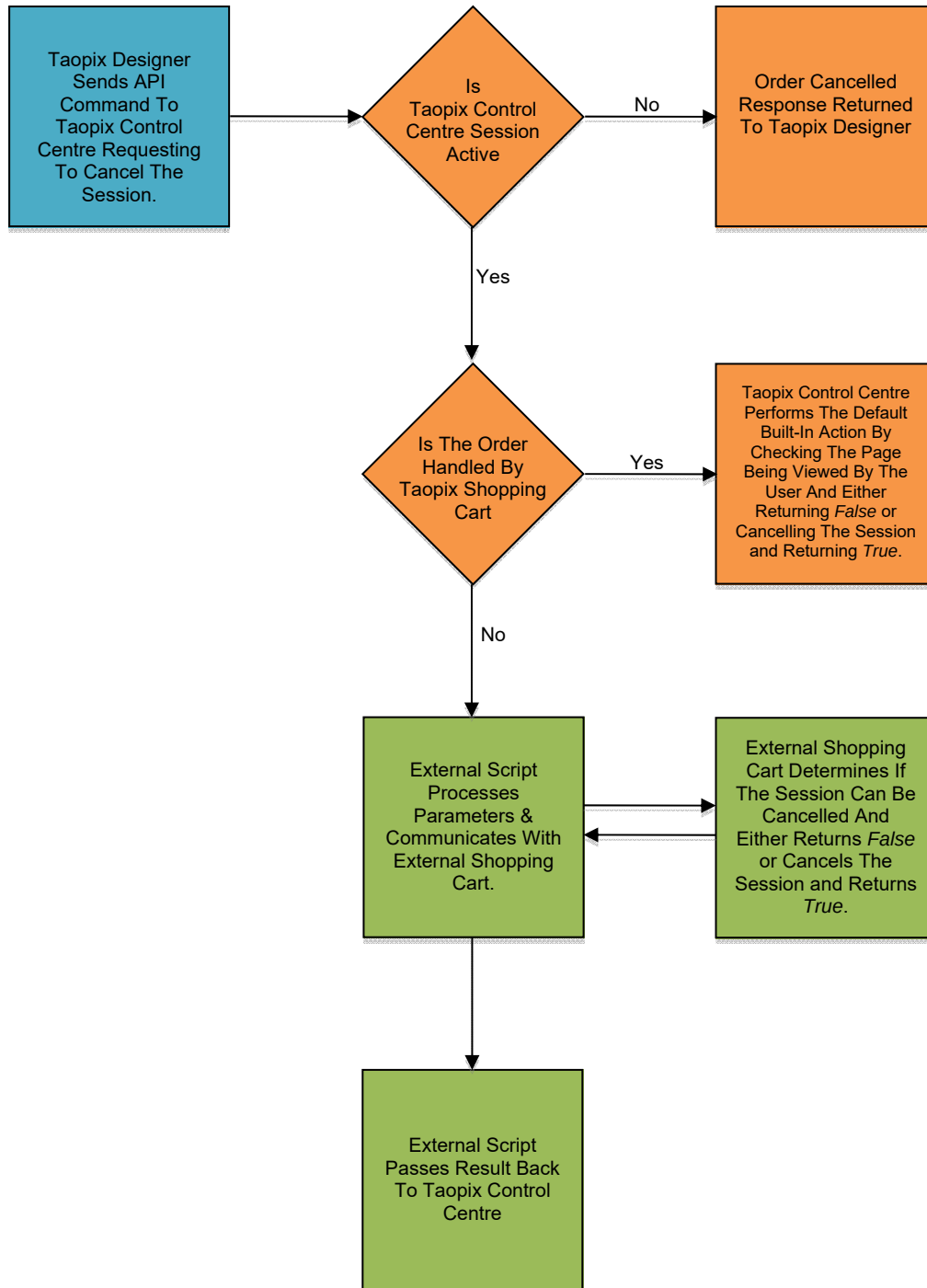
    If 'forcecancel' is equal to 0 then this is a request asking if the session can be cancelled.
    The external shopping cart should be queried to see where it is in it's workflow.
    If it is on a payment gateway page then this function should return 'false' to warn the user that
    they should not cancel.
    However, if the user clicks Cancel again then another request will be sent with 'forcecancel' set
    to 1.
    */

    $postParamArray = Array('cmd' => 'cancelsession', 'taopixsessionid' => $pParamArray['sessionid']);
    $result = self::cURLPost(kExternalShoppingCartURL, $postParamArray);

    return $result;
}
```

The functions that the integration should perform at this stage are:

- The script should transmit at minimum the `sessionid` and `forcecancel` values to the external shopping cart.
- The external shopping cart should receive the `sessionid` and use it to identify it's own session.
- The external shopping cart should check the `forcecancel` value and if it is 1 cancel it's session and return `true`. If `forcecancel` is 0 then the external shopping cart should determine if it's session can be cancelled. One way to do this could be to check which page the user is on within the external shopping cart workflow. If they are on the payment gateway page then `false` could be returned to warn the user that they should not cancel otherwise the session should be cancelled and `true` returned. However, the user could still decide to cancel in which case another API call would be received with `forcecancel` set to 1.



OPENING EXISTING PROJECTS IN ONLINE DESIGNER

When a customer attempts to re-open an existing project, the Taopix Online Designer checks to see if the customer has ever attempted to order the project in the shopping cart. If the Taopix Online Designer thinks that a shopping cart session may already be in progress it will also call the cancel session workflow as described above.

RESTARTING AN ORDER

The normal order workflow involves the customer uploading or saving to disk / burning their order data to CD/DVD directly after completing the shopping cart. However, if this process cannot be completed immediately or if the data needs to be sent again the customer must be able to complete the process at a later date. Taopix Control Centre will handle the initial processing of this request and if necessary it will call the `checkPageCount` function within the external script `EDL_ExternalShoppingCart.php`.

The purpose of this function is to make sure that the project that the customer is about to upload is acceptable and has not been modified in such a way that the cost would be effected (ie: product layout changed, page count changed etc...).

The input to the function is a single PHP associative array that receives the following parameters:

apiversion	integer	Taopix Designer API version
languagecode	varchar(50)	Taopix Designer language code
ownercode	varchar(50)	Taopix System unique identifier
groupcode	varchar(50)	Taopix license key code
groupdata	varchar(50)	Taopix license key data
brandcode	varchar(50)	Taopix brand code
orderid	integer	The order header record id within the Taopix Control Centre database
ordernumber	varchar(50)	The order number from the Taopix Control Centre database
items[]	array	A PHP associate array that will contain one or more projects

Each item will contain the following values:

orderitemid	integer	The order item record id within the Taopix Control Centre database
uploadref	varchar(200)	The unique upload reference for the project
productcode	varchar(50)	The new product code
orderedproductcode	varchar(50)	The product code that was originally ordered
pagecount	integer	The new page count
orderedpagecount	integer	The page count that was originally ordered
canuploadproductcodeoverride	integer	The indicator from Taopix Production stating whether the product code can be overridden 0=no 1=yes
canuploadpagecountoverride	integer	The indicator from Taopix Production stating whether the page count can be overridden 0=no 1=yes
externalassets[]	array	A PHP associate array that will contain one or more external picture assets for the project
pictures[]	array	A PHP associate array that will contain one or more single prints or photo print picture data for projects using the single prints or photo print module.
calendarcustomisations[]	array	A PHP associate array that will contain one or more calendar customisation data for projects using the calendar module.

The output of this function is a PHP string that is either empty to indicate that everything is okay, the pre-defined error `PRODUCTCODEMISMATCH` or a localised error message that will be displayed by Taopix Designer.

```
function checkPageCount($pParamArray)
{
    $result = '';

    $itemsArray = $pParamArray['items'];
    $itemCount = count($itemsArray);

    for ($i = 0; $i < $itemCount; $i++)
    {
        $theItem = &$itemsArray[$i];
    }

    return $result;
}
```

The functions that the integration should perform at this stage will depend on what validation is required for the integration. As an example, if the order has been placed using the Taopix shopping cart the following tests would be performed:

1. If `canuploadproductcodeoverride` is zero then the two product codes must match otherwise an error is returned.
2. If the page counts are different and `canuploadpagecountoverride` is zero then Taopix Control Centre checks the cover that was ordered to make sure it is available for the new page count and that the paper price has not increased.

A basic example of performing this validation is as follows:

```
function checkPageCount($pParamArray)
{
    $result = '';

    /*
    here is where the external shopping cart would check to see if any major changes have been to the
    project configuration since ordering
    */

    $itemsArray = $pParamArray['items'];
    $itemCount = count($itemsArray);

    for ($i = 0; $i < $itemCount; $i++)
    {
        $theItem = &$itemsArray[$i];

        /*
        check for a change in the product code
        */
        if (($theItem['productcode'] != $theItem['orderedproductcode']) &&
($theItem['canuploadproductcodeoverride'] == 0))
        {
            $result = 'PRODUCTCODEMISMATCH';
            break;
        }

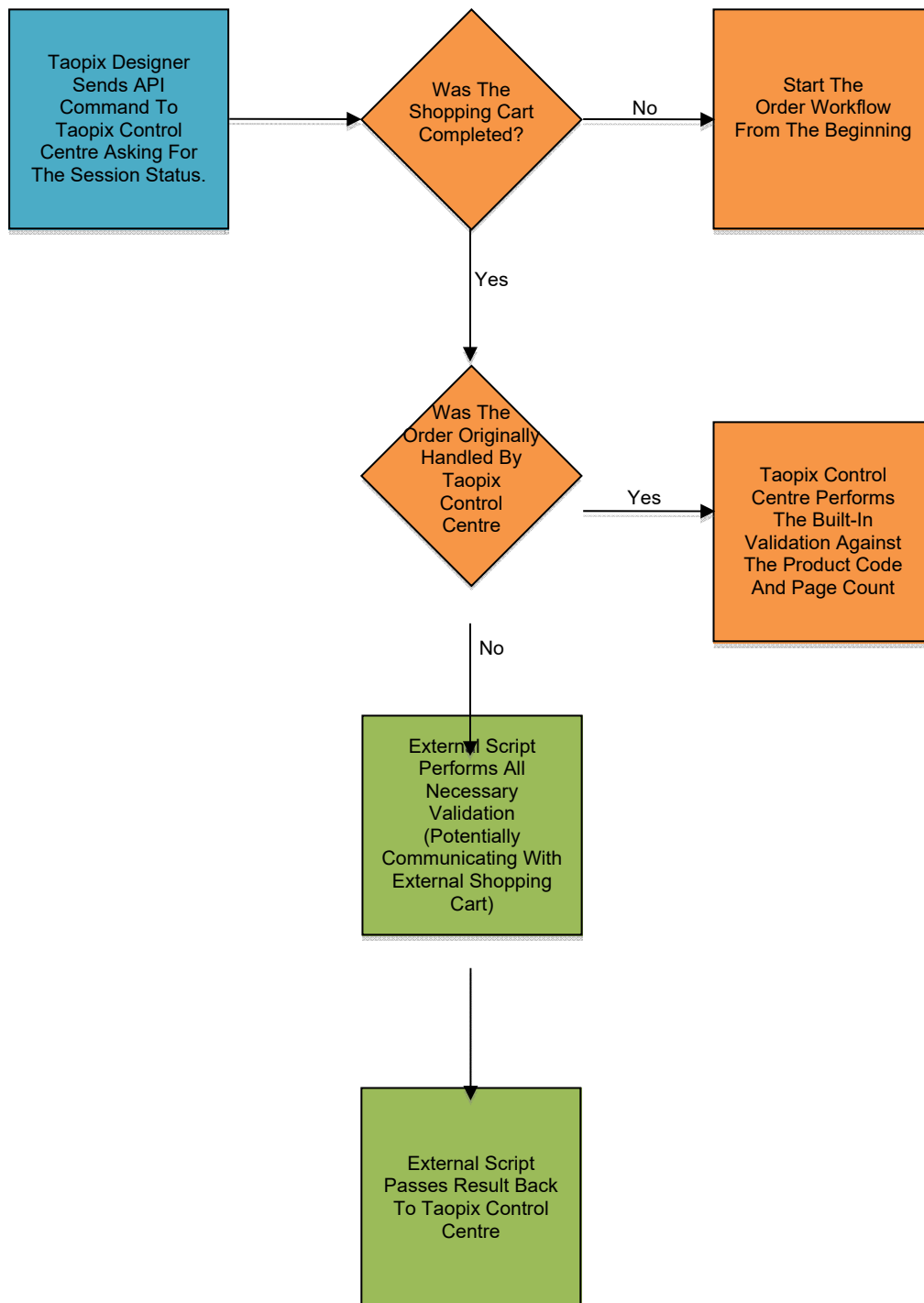
        /*
        check for a change in the page count
        */
        if (($theItem['pagecount'] > $theItem['orderedpagecount']) &&
($theItem['canuploadpagecountoverride'] == 0))
        {
            $result = 'The maximum number of pages is ' . $theItem['orderedpagecount'] . '.';
            break;
        }
    }

    return $result;
}
```

NOTES

1. If no error is returned from the validation, Taopix Control Centre automatically updates its database to reflect the new product code and page count. Depending on the integration it may be necessary for the external shopping cart to also update its own databases.
2. When the order has been placed using the Taopix shopping cart it also updates some of the order values based on the new product code and page count. This is not performed for external shopping cart orders.

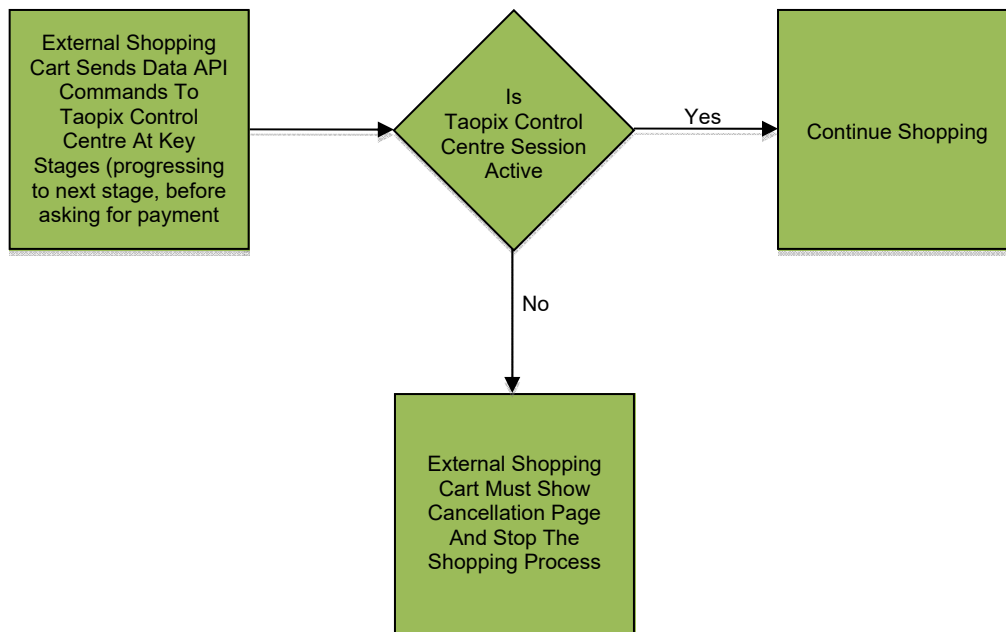
Workflow Diagram



EXTERNAL SHOPPING CART CHECK SESSION STATUS

When key stages are reached within the external shopping cart (before taking payment etc...) it is recommended that the external cart checks that status of the Taopix Control Centre session to make sure that it is still active. This process can be performed via the Data API's which are described later in this document.

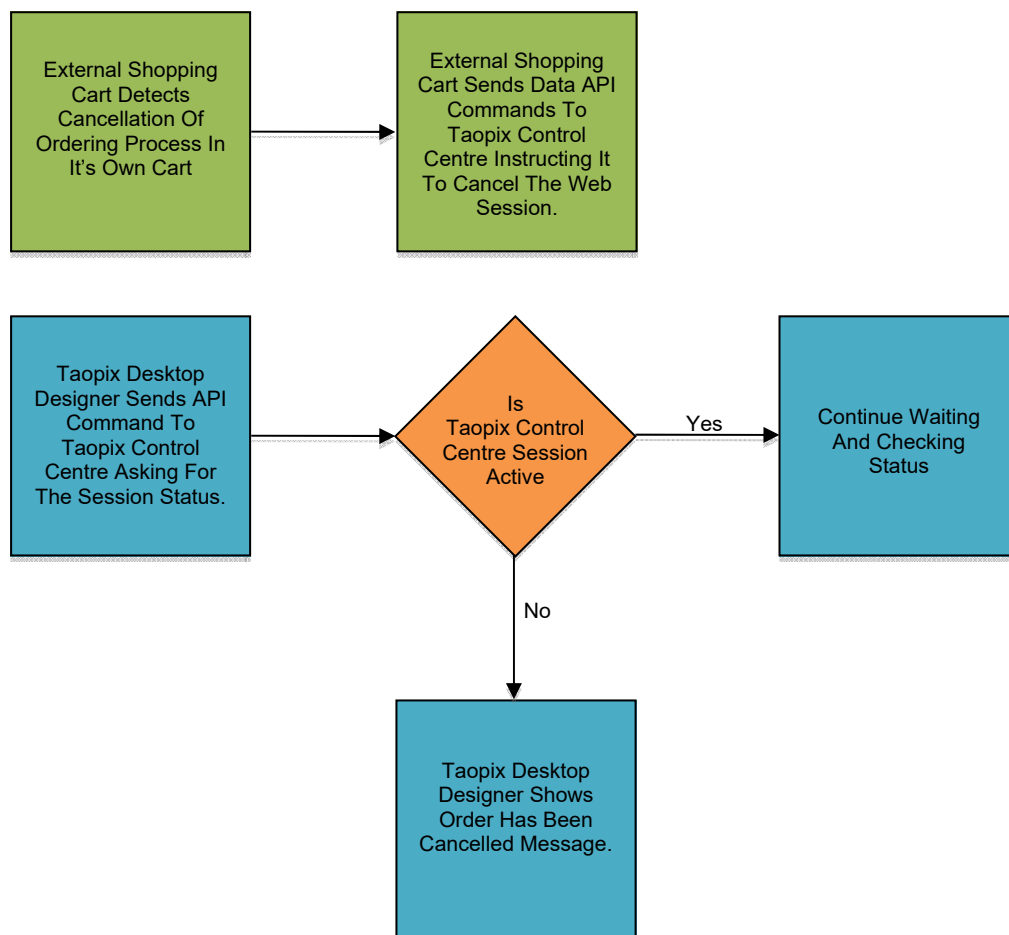
Workflow Diagram



EXTERNAL SHOPPING CART CANCELLATION

If the customer ends their shopping cart experience before the order is completed it is recommended that the external shopping cart cancels the Taopix Control Centre session. If the Taopix Desktop Designer is being used then the customer will also be notified. This process can be performed via the Data API's which are described later in this document.

Workflow Diagram



COMMUNICATING WITH TAOPIX CONTROL CENTRE

An external shopping cart or other business process may have a need to communicate with Taopix Control Centre for one of many different reasons such as checking a session status, cancelling a session, inserting an order into the Taopix Control Centre database or updating status information. In all situations this can be performed via the Taopix Control Centre Data API.

Please refer to the Taopix Data API v9.0.0 documentation for more details on how to insert an external shopping cart order back into Taopix Control Centre so that it can be processed by the Taopix Production client.

CURRENT LIMITATIONS

The following limitations will be addressed based on licensee feedback and requirements:

1. **Real-Time / Live Pricing**
Currently, the real-time / live pricing within Taopix Designer version 3 works by retrieving information from the Taopix pricing tables. This means that there is a requirement to configure pricing within Taopix Control Centre as well as the external cart. Future versions of the API's may support the retrieval of this information from the external shopping cart.
2. **Multi-Site Support**
Currently, the external shopping cart feature does not support the Taopix Multi-site workflow.
3. **Collect From Store Support**
Currently, the external shopping cart feature does not support the Taopix Collect-From-Store workflow. The store locator functionality can be provided by the external shopping cart but currently there is no support to push the order down the collect-from-store workflow instead of the standard direct shipping workflow.
4. **Offline Workflow**
It is currently not possible to use the offline workflow with an external shopping cart.